



How to apply Amdahl's law to multithreaded multicore processors



James Nutaro^{a,*}, Bernard Zeigler^b

^a Oak Ridge National Laboratory, PO Box 2008, MS6085, Oak Ridge, TN 37831-6085, United States

^b RTSync Corp. and Arizona Center for Integrative Modeling and Simulation, 4425 E Agave Rd., Suite 106, Phoenix, AZ 85044, United States

HIGHLIGHTS

- Models of speedup should separately account for memory and computational parts of a workload.
- Omitting memory access obscures the source of speedup and overstates the effect of multiple cores.
- Explicit accounting for memory access and computation clarifies and simplifies the model.

ARTICLE INFO

Article history:

Received 3 November 2015

Received in revised form

6 December 2016

Accepted 9 March 2017

Available online 4 April 2017

Keywords:

Multicore platforms

Speedup models

Amdahl's law

ABSTRACT

In this paper we comment on a recent article on Amdahl's law for multithreaded multicore processors. We propose that models for predicting speedup in such systems should explicitly separate the memory and computational parts of a workload. On the one hand, not doing so can lead to speedup calculations with more than one possible result. We show this for a simple example which exploits cache affinity between two sequential tasks. On the other hand, we show that explicit separation of the workload in this case leads to a unique result. We contend that decomposition of workloads into computational and memory parts can resolve similar ambiguity in general and is fundamental since it is apparent in the Turing Machine model of computation. Further research into this separation in the formulation of Amdahl's law for modern architectures is advocated.

© 2017 Elsevier Inc. All rights reserved.

A recent article by [1] has proposed a model for the speedup that can be achieved by multithreaded and multicore processors. One motivation for this model is that Amdahl's law was originally formulated for architectures that did not feature strong interactions between multiple threads via a shared memory cache. This suggests that Amdahl's law as usually stated is insufficient to explain the speedup of modern multithreaded and multicore systems. Indeed, the paper provides several important instances of where new insights are gained with a new model. One of these cases is Result 3, which states that "super linear speedup is achievable when threads exhibit strong cache affinity, the workload is strong memory-bound, and there is no resource contention among threads". Unfortunately, the definition of super linearity is not explicitly given, but the results in the paper imply it to mean "a speedup greater than the number of threads". In the following, we restrict the use of "super linear" to this definition.

There appears to be an implicit assumption in Che and Nguyen's queueing model that the instructions within a thread exhibit high

cache affinity. If this were not the case, then each instruction within a thread would be unlikely to find its data in the cache regardless of the activity of other threads and super linear speedup induced by cache affinity between threads could not appear. One artifact of this assumption is that time spent within a thread to access memory does not appear in the model except where one thread could evict data required by another. This omission is significant because it introduces a second source of super linear speedup. Specifically, if cache affinity is not present within a thread, then super linear speedup is achievable by improving that thread's cache affinity. The speedup achieved in this way appears to be super linear because the time spent fetching data from main memory is not considered part of the computational workload. On the other hand, if the workload includes time to access memory then the model can explain both sources of super linear speedup and begins to resemble Amdahl's law.

To show how including memory access in the workload leads to super linear speedup, we create a corollary to Result 3 stating that if cache affinity is not present within a thread, then super linear speedup is achievable without increasing the number of threads. Specifically we demonstrate such speedup in the case of just one thread. Consider a program that performs two distinct but

* Corresponding author.

E-mail address: nutarojj@ornl.gov (J. Nutaro).

unrelated operations on two series of data each of length d . A time t is needed for each operation and t units of time are needed to fetch the datum to be operated upon (setting these times equal for convenience only). We can calculate the execution time of this program run with a single thread to be $4dt$ because for each of the d data the program must (i) fetch the datum for operation one, (ii) execute that operation, (iii) fetch the datum for operation two, and (iv) execute that operation.

Now suppose we can arrange this program such that the two operations use data on the same cache line; that is, just one fetch is required because the second operation always finds the data it needs in the cache. This affinity between the two operations reduces the execution time to $3dt$ and yields super linear speedup. Specifically, this single threaded program achieves a speedup of 1.25 over the single threaded program without the cache optimization. We have achieved this speedup without requiring additional cores or hardware threads and so proved the corollary.

Unfortunately, adding memory access time to Che and Nguyen's queueing model creates ambiguity when a thread having only partial cache affinity within itself interacts with other threads via the cache. To illustrate this ambiguity, we assign the separate operations in the above calculation to a pair of hardware threads residing on a single core. In this multithreaded program, each thread always finds its data in memory following the initial fetch. This eliminates the cost of memory fetches but the operations themselves share a single hardware core, hence the execution time is $2dt$. There are at least two possible ways of calculating speedup. In the first case, we take advantage of cache affinity after parallelizing the program. In this case, the sequential program needs $4dt$ units of time to execute and the speedup is 2. In the second case, we take advantage of cache affinity before parallelizing the program. Now, as demonstrated above, the single thread only needs $3dt$ time and the speedup is only 1.5. This apparent ambiguity does not invalidate the general results of the paper, but it does raise questions of how the model should be applied in situations where complete cache affinity within a thread is not a valid approximation.

In this particular example, the queueing theory model obscures rather than illuminates the important role of cache management, and we believe we can shine light on the result with a more direct model; specifically, Amdahl's law with an explicit accounting for the memory and computational parts of the workload. This model has n processing units and a workload with two parts: the memory-related part m and the computation-related part c so that the sequential workload is $m + c$. The effect of cache affinity (between threads or operations) is to reduce the memory workload. In our example above, $c = 2dt$ while m was reduced from $2dt$ to dt . Thus the speedup is uniquely calculated as

$$(2dt + 2dt)/(2dt + dt) = 4/3 = 1.25.$$

When there is no cache affinity the speedup upper bound achievable by n threads is $(m + c)/n$ because the cache cannot eliminate the memory related part of the workload; this is true for both the queueing model and our much simpler model and is independent of how the threads are distributed in cores. In the best case, cache affinity sets $m = 0$ and, assuming the computational workload is unaffected, the speedup bound achievable by n threads becomes c/n , which relative to the sequential workload without cache affinity is

$$((m + c)/n)/(c/n) = m/c + 1. \quad (1)$$

This ratio is super linear by a factor that grows with the ratio of the memory workload (before elimination) to the computational workload. By breaking the problem into two parts by first accounting for cache affinity to reduce the workload from $m + c$ to c and second asking about the speedup obtainable for the reduced

workload by using n threads we obtain a unique result in the multithreaded case.

The article in question identifies threads as processing units and gives credit to cache affinity and the presence of threads in multicore architectures for achieving super linear speedup. However, the model we just discussed is agnostic to the nature of the processing units and memory-related workload reduction. Indeed, we can confirm that this is the case by formulating the problem in a fundamental manner with a Turing machine (TM) interpretation. In brief, consider a parallel system of n TMs each of which works on a single square of tape. To perform the same computation, a single (sequential) TM must move itself in succession to each square. Thus the workload performed by the latter appears to have n more move operations than the original parallel system and this extra memory associated work plays the same role as the sequential workload above. Indeed, we get the same result as in Eq. (1). However, in this formulation, the set-up work to get the parallel system into the right initial configuration (moving each of the TMs to its place) is apparent—taking it into account, one must add $O(n^2)$ to the parallel computation, thus nullifying (for large n) the apparent super linearity due to a reduced memory workload.

In summary, applications of the queueing model proposed by Che and Nguyen should proceed from a careful statement about when cache optimizations are applied to the workload. As we have demonstrated, the speedup results obtained with the model can depend substantially on this assumption. This apparent ambiguity was resolved with an explicit separation of computational and memory bound workloads along the lines illustrated in our simple model. We suggest that such a separation could help to resolve ambiguity and offer insights into a broader range of questions concerning threads and cache management in modern computer systems, offering an area for future research. While it is impractical to use explicit sequences of instructions to model speedup on average, it may be possible to extend the existing, probabilistic model for between thread interactions via the cache to also account for within thread caching behavior; we leave this for future research. A reviewer raised the question of whether we are opening up workload characterization to perhaps an indefinite number of different types by adding the memory workload concept. In response, we showed that the computation/memory distinction appears to be fundamental and readily apparent in the universally accepted Turing Machine model of computation.

References

- [1] Hao Che, Minh Nguyen, Amdahl's law for multithreaded multicore processors, *J. Parallel Distrib. Comput.* 74 (2014) 3056–3069.



James Nutaro is a Senior Research Staff member at Oak Ridge National Laboratory. He has extensive experience in M&S and systems modeling in both defense and commercial domains. He has applied M&S techniques for design, analysis, and testing in diverse enterprises including missile systems, space systems, communications, manufacturing, electrical power, disease processes, and high performance computing technology. He is also the author of *Building Software for Simulation: Theory and Algorithms*, with Applications in C++.



Bernard P. Zeigler, Chief Scientist of RTSync Corp., and Emeritus Professor from Arizona Center for Integrative Modeling and Simulation is internationally known for his seminal contributions in M&S theory, and has published several books including *Theory of Modeling and Simulation*, IEEE named him a Fellow of the IEEE for his invention of the Discrete Event System Specification (DEVS).